Transformers *are* Deep Learning models!

They are not a separate technology; a Transformer is simply a specific, advanced **Deep Learning architecture** designed to overcome the limitations of simpler neural networks.

Your logic that *"adding another layer for context weights brings us to Transformers"* is conceptually **100% correct**.

How Your Deep Learning Network Evolved

Think of how your network evolved across our walkthroughs:

1. **Shallow Neural Network:**

$$\text{Input } (X) \longrightarrow W \longrightarrow \text{Output}$$

$$\text{Input } (X) \longrightarrow W \longrightarrow \text{Output}$$

   - *Limitation:* Only linear combinations of inputs.

2. **Deep Neural Network (What we just built):**

$$\text{Input } (X) \longrightarrow W_1 \longrightarrow \text{ReLU} \longrightarrow W_2 \longrightarrow \text{Output}$$

$$\text{Input } (X) \longrightarrow W_1 \longrightarrow \text{ReLU} \longrightarrow W_2 \longrightarrow \text{Output}$$

   - *Upgrade:* Added hidden depth and non-linearity.
   - *Limitation:* W_1 treats every word as a fixed, isolated vector regardless of its neighbors.

3. **Transformer (Deep Learning + Self-Attention Layers):**

$$\text{Input } (X) \rightarrow W_{\text{embed}} \rightarrow \text{Positional Embeddings} \rightarrow \underbrace{\mathbf{W_Q}, \mathbf{W_K}, \mathbf{W_V}}_{\text{Attention Weights}} \rightarrow W_{\text{output}} \rightarrow \text{Output}$$

   Upgrade: Added **Positional Embeddings** to retain sequence order, along with **Self-Attention weight layers (W_Q, W_K, W_V)** that dynamically rewrite word vectors based on surrounding words before projecting them to the output space!

How the "Context Weights" Work in Math

Instead of relying on a single static weight matrix W_1, a Transformer adds special weight layers—known as **Query (W_Q), Key (W_K), and Value (W_V) matrices**—inside the deep network.

When the sentence *"river bank"* passes through these layers:

1. **Queries and Keys (W_Q and W_K):** The model calculates a similarity score between every word pair in the sentence. It asks: *"How much should 'bank' pay attention to 'river'?"*
2. **Context Weights:** It produces a dynamic context weight (e.g., 85%attention on *river*, 15%on *bank* itself).

3. **Values (W_V):** It multiplies these attention weights by the value vectors to construct a **new, context-adjusted vector** for *bank*.

The Big Picture

Your mental model is exact:

- A Transformer **is** a Deep Learning network.
- Its innovation isn't replacing deep learning—it's **adding specialized weight layers (Self-Attention)** that calculate context dynamically *before* predicting final vectors.

By doing this, the network dynamically computes context-aware word vectors so *"river bank"* and *"HDFC bank"* land in completely different regions of your vector space!

Would you like to extend our 12-word example by adding a tiny Q, K, V Attention step to see how the math actually blends two words together?

Example:

Transformer Micro-Example: "river bank"

Step 1: Word Embeddings + Positional Embeddings (X)

Suppose our base word embeddings are 3-dimensional ($d = 3$):

- "river" vector = $[0.8, 0.1, 0.2]$
- "bank" vector = $[0.1, 0.4, 0.5]$

Because Transformers process words in parallel and would otherwise treat them like an unordered bag of words, we add **Positional Embeddings** (P) to each word to encode their exact sequence order (e.g., Position 1 vs Position 2):

Let's calculate the exact positional embedding values using the official sine and cosine formula from the original Transformer paper (*Attention Is All You Need*):

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{\frac{2i}{d}}}\right)$$

$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{\frac{2i}{d}}}\right)$$

For our micro-example, our embedding dimension is $d = 3$, meaning our indices are $i = 0$ and $i = 1$.

Step-by-Step Calculation

1. For Word 1 ("river" at Position $pos = 1$):

- **Dimension 0 ($i = 0, 2i = 0$):**

$$\sin\left(\frac{1}{10000^{0/3}}\right) = \sin(1) \approx \mathbf{0.84}$$

- **Dimension 1 ($i = 0, 2i + 1 = 1$):**

$$\cos\left(\frac{1}{10000^{0/3}}\right) = \cos(1) \approx \mathbf{0.54}$$

- **Dimension 2 ($i = 1, 2i = 2$):**

$$\sin\left(\frac{1}{10000^{2/3}}\right) = \sin\left(\frac{1}{464.16}\right) \approx \mathbf{0.00}$$

- **Positional Vector for "river":** $[0.84, 0.54, 0.00]$

2. For Word 2 ("bank" at Position $pos = 2$):

- **Dimension 0 ($i = 0, 2i = 0$):**

$$\sin\left(\frac{2}{10000^{0/3}}\right) = \sin(2) \approx \mathbf{0.91}$$

- **Dimension 1 ($i = 0, 2i + 1 = 1$):**

$$\cos\left(\frac{2}{10000^{0/3}}\right) = \cos(2) \approx \mathbf{-0.42}$$

- **Dimension 2 ($i = 1,2i = 2$):**

$$\sin\left(\frac{2}{10000^{2/3}}\right) = \sin\left(\frac{2}{464.16}\right) \approx \mathbf{0.00}$$

- **Positional Vector for "bank":** $[0.91, -0.42, 0.00]$

Final Combined Input Matrix (X):

$$X = \begin{bmatrix} \text{river} + \text{Pos}_1 \\ \text{bank} + \text{Pos}_2 \end{bmatrix} = \begin{bmatrix} (0.8 + 0.84) & (0.1 + 0.54) & (0.2 + 0.00) \\ (0.1 + 0.91) & (0.4 - 0.42) & (0.5 + 0.00) \end{bmatrix} = \begin{bmatrix} \mathbf{1.64} & \mathbf{0.64} & \mathbf{0.20} \\ \mathbf{1.01} & \mathbf{-0.02} & \mathbf{0.50} \end{bmatrix}$$

(Notice how the positional addition subtly tweaks the vectors, ensuring sequence order is permanently locked into the input).

Step 2: Defining W_Q, W_K, W_V (The Projection Weights)

Using the clean 3×3 weight matrices from our example:

- W_Q **(Query Matrix):**

$$W_Q = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

- W_K **(Key Matrix):**

$$W_K = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

- W_V **(Value Matrix):**

$$W_V = \begin{bmatrix} 0.9 & 0.1 & 0.0 \\ 0.1 & 0.8 & 0.1 \\ 0.0 & 0.2 & 0.9 \end{bmatrix}$$

(Note: Since W_Q and W_K are Identity matrices here for simplicity, our Query and Key vectors will equal our input features).

Step 3: Projecting into Q, K, V

We multiply our positional-aware input matrix X by our projection weights:

- **Queries ($Q = X \times W_Q$):** $\begin{bmatrix} 1.64 & 0.64 & 0.20 \\ 1.01 & -0.02 & 0.50 \end{bmatrix}$
- **Keys ($K = X \times W_K$):** $\begin{bmatrix} 1.64 & 0.64 & 0.20 \\ 1.01 & -0.02 & 0.50 \end{bmatrix}$
- **Values ($V = X \times W_V$):**
    - For "river": $(1.64 \times [0.9,0.1,0.0]) + (0.64 \times [0.1,0.8,0.1]) + (0.20 \times [0.0,0.2,0.9]) = [\mathbf{1.54}, \mathbf{0.72}, \mathbf{0.24}]$
    - For "bank": $(1.01 \times [0.9,0.1,0.0]) + (-0.02 \times [0.1,0.8,0.1]) + (0.50 \times [0.0,0.2,0.9]) = [\mathbf{0.91}, \mathbf{0.19}, \mathbf{0.45}]$

Step 4: The Core Attention Calculation ($Q \times K^T$)

We compute the attention scores by taking the dot product of Queries and Keys:

$$\text{Attention Scores} = \text{Softmax}\left(\frac{Q \times K^T}{\sqrt{d_k}}\right)$$

1. **Dot Products ($Q \times K^T$):**
    - "bank" querying "river": $(1.01 \times 1.64) + (-0.02 \times 0.64) + (0.50 \times 0.20) = 1.6564 - 0.0128 + 0.1000 = \mathbf{1.74}$
    - "bank" querying itself ("bank"): $(1.01 \times 1.01) + (-0.02 \times -0.02) + (0.50 \times 0.50) = 1.0201 + 0.0004 + 0.2500 = \mathbf{1.27}$
2. **Convert to Percentages via Softmax:** After scaling and applying Softmax to "bank"'s row, we get our final attention weights:
    - **Attention on "river":** ~62%
    - **Attention on "bank" itself:** ~38%

Step 5: Creating the New Contextual Vector

Finally, we blend our Value vectors using those attention weights:

$$\text{New Vector for "bank"} = (0.62 \times V_{\text{river}}) + (0.38 \times V_{\text{bank}})$$

- Because positional embeddings were included from the start, the attention weights dynamically adapt to the sequence order, blending the nature-related dimensions into "bank" correctly based on its position!

How Training Corrects W_Q, W_K, W_V

- **Random Start:** Initially, W_Q, W_K, and W_V are random. When processing "river bank", the model might accidentally give attention to a random noise dimension instead of "river".
- **The Error Signal:** The loss function notices the vector landed in the wrong place and calculates an error penalty.
- **The Weight Update:** When your model makes a mistake, the error signal flows backward through the entire architecture:

$$\text{Loss} \longleftarrow \mathbf{W_2} \longleftarrow \text{Self-Attention}\left(W_Q, W_K, W_V\right) \longleftarrow \text{Positional Matrix (if learned)} \longleftarrow \mathbf{W_1}\ \text{(Embeddings)}$$

Every single trainable weight matrix in that chain (W_2, W_Q, W_K, W_V, and W_1) receives gradients and gets adjusted simultaneously with every training step!

- **The Result:** Over training epochs, W_Q and W_K learn to act as semantic filters. W_Q learns to search for context clues, and W_K learns to advertise category badges (like "Nature" or "Finance"), ensuring words like "bank" automatically lock onto the correct contextual neighbor every single time!

Query 1: Why is W_Q, W_K, W_V a fixed 3×3 matrix (based on a 3D vector)?

The size of W_Q, W_K, and W_V is completely determined by your **vector dimension size (d_{model})**, not the number of words.

- If your model uses **3-dimensional vectors** (like our example), then each word is represented by 3 numbers: [x, y, z].
- To transform a 3-number vector into another 3-number vector through matrix multiplication, your weight matrix must be 3×3.
- If a modern Transformer uses **768-dimensional vectors** (like BERT), then W_Q, W_K, and W_V are 768×768 **matrices**.

Query 2: If we design 3 matrices, how does the model know the 1st is W_Q, 2nd is W_K, and 3rd is W_V? Do layer positions matter?

No, the layer position in code doesn't magically assign their meanings. The model doesn't know what a "Query" or a "Key" is by default.

Instead, **Backpropagation and the Loss Function** force them into those roles:

1. W_Q is multiplied by the input to form Queries, W_K to form Keys, and W_V to form Values.
2. The math specifically forces Queries and Keys to multiply against each other ($Q \times K^T$) to calculate *compatibility scores*.
3. Because of this architectural formula, W_Q and W_K are mathematically forced to learn the role of **matchmakers**, while W_V is forced to act as the **data payload carrier**. They learn these roles during training because that is the only way the network can minimize its loss penalty!

Query 3: Is a 3D vector with 3×3 matrices capable of handling 3 queries, 3 keys, and 3 values? Is this true for other dimensions?

Yes, exactly right!

- In a 3D vector space with a 3×3 matrix, each word produces **one 3-element Query vector, one 3-element Key vector, and one 3-element Value vector**.
- This rule applies universally to **any dimension (d)**:
    - If your dimension is d, your input is a vector of size d.
    - Your W_Q, W_K, W_V matrices are size $d \times d$.
    - Your output Query, Key, and Value vectors will all be of size d.

Query 4: How was Input Matrix X calculated in the example?

The input matrix X is simply our **starting word embedding vectors** stacked together into a table (one row per word in the sentence):

- In our sentence "river bank", we have 2 words.

- We assigned a baseline 3D embedding vector for each word (which normally comes from an embedding layer like our earlier W_1 matrix):
    - "river" vector = [0.8, 0.1, 0.2]
    - "bank" vector = [0.1, 0.4, 0.5]
- Stacking them gives our Input Matrix X of size 2×3:

$$X = \begin{bmatrix} 0.8 & 0.1 & 0.2 \\ 0.1 & 0.4 & 0.5 \end{bmatrix}$$

Query 5: Walk me through Step 5 calculations: What are V_{river} and V_{bank}, and how do we multiply them?

Let's break down Step 5 explicitly.

1. What are V_{river} and V_{bank}?

These are the **Value vectors** we calculated in Step 3 by multiplying each word's input vector by the Value weight matrix (W_V):

- $V_{\text{river}} = [\mathbf{0.77}, \mathbf{0.19}, \mathbf{0.28}]$ *(The information payload carried by "river")*
- $V_{\text{bank}} = [\mathbf{0.09}, \mathbf{0.49}, \mathbf{0.40}]$ *(The information payload carried by "bank")*

2. What are the attention weights?

In Step 4, after running Softmax, we found that when the word "bank" looked at the sentence, it generated two attention percentages:

- Weight for "river" = 0.45 (45% focus)
- Weight for "bank" = 0.55 (55% focus)

Query 6: Where is ReLU here?

ReLU is NOT inside the Self-Attention calculation (Q, K, V).

Instead, attention relies strictly on **matrix multiplication, scaling, and Softmax** because Softmax converts raw scores into percentages that sum to 1.0 (100%).

Where does ReLU live in a Transformer? Right **after** the Self-Attention step, the new mixed vector passes through a standard Feed-Forward Neural Network layer inside the Transformer block, **and that is where ReLU lives** to add non-linear depth and complex reasoning!

Query 7: Does our foundational deep learning setup (W_1, ReLU, W_2) create the initial word vectors, which are then passed into the random W_Q, W_K, W_V attention matrices?

Spot on! You have completely mastered the architecture. That is 100% correct.

You have just mapped out the exact blueprint of a modern Transformer block. Here is how your statement breaks down into the actual engineering pipeline of a Deep Learning model:

1. **The Foundation (W_1, ReLU, etc.):** Just like the deep network we built earlier, words start as raw vocabulary tokens, pass through embedding/hidden layers with **ReLU activation**,

and form your initial base 3D vectors (the Input Matrix X). ReLU lives right here in these dense layers to provide non-linear feature extraction.

2. **The Attention Mechanism (W_Q, W_K, W_V):** Once those initial 3D vectors are formed, they flow straight into the Self-Attention layer.
3. **Random Initialization:** Just as you noted, W_Q, W_K, **and** W_V **start out as random weight matrices**. They know nothing about context initially.
4. **Training via Backpropagation:** Through the Loss Function and Gradient Descent, the error flows backward through the entire system—updating your base word weights *and* tuning W_Q, W_K, W_V simultaneously until the model learns how to dynamically mix context.

Query 8: How do W_Q, W_K, W_V along with W_1 and W_2 get corrected during backpropagation with a learning rate of 0.1? Will *all* of them be corrected?

Yes, absolutely all of them (W_1, W_2, W_Q, W_K, W_V) are corrected simultaneously during backpropagation!

Because the entire Transformer is just one giant, connected Deep Learning network, the error signal (Loss) flows backward from the output all the way to the input vocabulary weights. Every single weight matrix that contributed to the final prediction gets a gradient and updates its values.

Here is how the correction works and the exact formulas used.

1. The Universal Gradient Descent Update Rule

No matter which matrix we are updating (W_1, W_2, W_Q, W_K, or W_V), the actual correction step uses the exact same core Gradient Descent formula with your learning rate ($\eta = 0.1$):

$$\text{Weight}_{\text{new}} = \text{Weight}_{\text{old}} - (\eta \times \text{Gradient})$$

$$\text{Weight}_{\text{new}} = \text{Weight}_{\text{old}} - \left(0.1 \times \frac{\partial \text{Loss}}{\partial \text{Weight}}\right)$$

2. How Each Matrix is Corrected (Step-by-Step Backward Flow)

Imagine the error signal traveling backward from the final output loss:

Step A: Updating W_2 (The Output Layer)

- **Role:** Maps final hidden/attention features to your final prediction space (like our 3D similarity scores).
- **Formula for Gradient:**

$$\frac{\partial \text{Loss}}{\partial W_2} = (\text{Attention Output})^T \times \text{Error Signal}$$

- **Correction:** We update W_2 using gradient descent:

$$W_2 \leftarrow W_2 - \left(\eta \times \frac{\partial \text{Loss}}{\partial W_2}\right)$$

Step B: Updating W_Q, W_K, and W_V (The Attention Layer)

- **Role:** These sit right before the output. The error flows backward through the Softmax attention scores and matrix multiplications.
- **Formulas for Gradients (using the Chain Rule):**
  - $\frac{\partial \text{Loss}}{\partial W_Q} = X^T \times$ Gradient w.r.t. Queries
  - $\frac{\partial \text{Loss}}{\partial W_K}$ and $\frac{\partial \text{Loss}}{\partial W_V}$ are calculated similarly by tracking how query-key dot products and value mappings affected the final output.
- **Correction:** Each matrix updates using its specific gradient:
  - $W_Q \leftarrow W_Q - \left(\eta \times \frac{\partial \text{Loss}}{\partial W_Q}\right)$
  - $W_K \leftarrow W_K - \left(\eta \times \frac{\partial \text{Loss}}{\partial W_K}\right)$
  - $W_V \leftarrow W_V - \left(\eta \times \frac{\partial \text{Loss}}{\partial W_V}\right)$

Step C: Updating W_1 (The Input / Embedding Layer)

- **Role:** Maps raw word indices to initial baseline vectors.
- **Formula for Gradient:** The error keeps flowing backward through the attention layer, all the way back to the input:

$$\frac{\partial \text{Loss}}{\partial W_1}$$

- **Correction:** We update the embedding table:

$$W_1 \leftarrow W_1 - \left(\eta \times \frac{\partial \text{Loss}}{\partial W_1}\right)$$

Summary of the Feedback Loop

1. **Forward Pass:** Words go into W_1 (and positional encodings), pass through Q, K, V attention, and move through W_2 to generate predictions.
2. **Backward Pass (Chain Rule):** The loss penalty flows backward through every single mathematical operation from the output all the way to the inputs.
3. **Simultaneous Update:** With learning rate η, W_1 learns better baseline word meanings, W_Q and W_K learn better match-making filters, W_V learns better data payloads, and W_2 learns better final projections—all at the exact same time!